

2020

1.

Dossier des tests unitaires



MaraisRsense

Nom étudiant

Sommaire

Table des matières

1.NOM CLASSE	3
1.1.HISTORIQUE DES VERSIONS	3
1.2.PLAN DE TEST	4
1.3.FICHES DE TEST.....	5
<i>1.3.1.1^{er} test.....</i>	<i>5</i>
1.4.COMPTE RENDU DE TEST.....	6
1.5.RÉSULTAT DE L'EXÉCUTION DU PROGRAMME DE TEST	7

2. *Nom classe*

2.1. Historique des versions

Version	Date	Classe/Module	Modification	Responsable
1.0	30/03/2026	Ccs811	Création du document	Côme PETIT

2.2. Plan de test

Test Unitaire Plan de test

Test					
Nom	TestCcs811		TU1	Type	Fonctionnel
Unité					
Nom	Ccs811	Type	Classe		
Réalisation					
Nom du testeur	Côme PETIT		Date	30/03/2026	
Nom du framework	unittest				

Objectifs

Tester le fonctionnement nominal des différentes méthodes de la classe Ccs811 pour la détection de CO2 et TVOC dans le projet MaraisRSense. Vérifier l'initialisation, la lecture/écriture I2C et les mesures simulées.

Environnement

- ✓ Fichiers : Ccs811.py, test_ccs811.py
- ✓ Python 3.x avec unittest et unittest.mock
- ✓ Simulation du bus I2C avec mocks (pas de matériel réel requis)

Procédure de test

- ❖ Lancer `python -m unittest test_ccs811`
- ❖ Contrôler les résultats dans la console

Résultats attendus

Ran 8 tests in X.XXXs
OK

Référence	Nom du test	Nom de l'unité	Description
TU1.1	TestReadReg	read_reg	Lecture registre I2C simulé
TU1.2	TestWriteReg	write_reg	Écriture registre I2C simulé
TU1.3	TestDataReadyTrue	data_ready	Données prêtes = True
TU1.4	TestDataReadyFalse	data_ready	Données prêtes = False
TU1.5	TestReadEco2Tvoc	read_eco2_tvoc	Lecture eCO2/TVOC simulés
TU1.6	TestCcs811InitOK	ccs811_init	Init OK avec capteur simulé

2.3. Fiches de test

2.3.1. 1^{er} test

Test Unitaire					
Fiche de test					
Test					
Nom	TestReadReg	Référence	TU1.1	Type	Fonctionnel
Unité					
Nom	read_reg	Type	Méthode		
Réalisation					
Nom du testeur	Côme PETIT		Date	30/03/2026	

Objectifs			
Vérifier que read_reg lit correctement un registre simulé via mock I2C.			
Étapes			
Pré-conditions	Post-conditions	Actions	Résultats attendus
Mock I2C configuré avec [0x81]	Valeur lue = [0x81]	capteur.read_reg(REG_HW_ID, 1)	VRAI
Mock I2C vérifié	Appel correct	Vérif appel readfrom_mem(0x5A, 0x20, 1)	VRAI

Test Unitaire Fiche de test					
Test					
Nom	TestWriteReg	Référence	TU1.2	Type	Fonctionnel
Unité					
Nom	write_reg	Type	Méthode		
Réalisation					
Nom du testeur	Côme PETIT		Date	30/03/2026	

Objectifs			
Vérifier que write_reg écrit correctement via mock I2C.			
Etapas			
Pré-conditions	Post-conditions	Actions	Résultats attendus
Mock I2C prêt	Appel writeto_mem(0x5A, 0x01, [0x10])	capteur.write_reg(REG_MEAS_MODE, [0x10])	VRAI

Test Unitaire <i>Fiche de test</i>					
Test					
Nom	TestDataReadyTrue	Référence	TU1.3	Type	Fonctionnel
Unité					
Nom	data_ready	Type	Méthode		
Réalisation					
Nom du testeur	Côme PETIT		Date	30/03/2026	

Objectifs			
Vérifier data_ready() retourne True quand status=0x08.			
Etapas			
Pré-conditions	Post-conditions	Actions	Résultats attendus
Mock read_reg=[0x08]	VRAI	capteur.data_ready() Retourne True	

Test Unitaire Fiche de test					
Test					
Nom	TestReadEco2Tvoc	Référence	TU1.5	Type	Fonctionnel
Unité					
Nom	read_eco2_tvoc	Type	Méthode		
Réalisation					
Nom du testeur	Côme PETIT		Date	30/03/2026	

Objectifs			
Vérifier décodage eCO2/TVOC à partir des 4 octets.			
Etapas			
Pré-conditions	Post-conditions	Actions	Résultats attendus
Mock read_reg=[0x04,0xD2, 0x00,0x64]	(1234, 100)	capteur.read_eco2_tvoc()	VRAI

Test Unitaire <i>Fiche de test</i>					
Test					
Nom	TestCcs811InitOK	Référence	TU1.6	Type	Fonctionnel
Unité					
Nom	ccs811_init	Type	Méthode		
Réalisation					
Nom du testeur	Côme PETIT		Date	30/03/2026	

Objectifs			
Etapas			
Pré-conditions	Post-conditions	Actions	Résultats attendus
Mock scan=[0x5A], HW_ID=[0x81]	Écriture APP_START + MEAS_MODE=0x10	capteur.ccs811_init()	VRAI

2.4. Compte rendu de test

Test Unitaire Compte rendu					
Test					
Nom	Ccs811	Référence	TU1	Type	Fonctionnel
Unité					
Nom	Ccs811	Type	classe		
Réalisation					
Nom du testeur	Côme PETIT		Date	30/03/2026	

Procédure du test		
Référence	Résultats attendus	Résultats obtenus
TU1.1	VRAI	VRAI
TU1.2	VRAI	VRAI
TU1.3	VRAI	VRAI
TU1.4	VRAI	VRAI
TU1.5	VRAI	VRAI
TU1.6	VRAI	VRAI
Nature des modifications (fichier source et/ou test)		

2.5. Résultat de l'exécution du programme de test

```
$ python -m unittest test_ccs811 -v
test_ccs811_init_capteur_absent
(test_ccs811.TestCcs811) ... ok
test_ccs811_init_hw_id_incorrect
(test_ccs811.TestCcs811) ... ok
test_ccs811_init_ok (test_ccs811.TestCcs811) ... ok
test_data_ready_false (test_ccs811.TestCcs811) ... ok
test_data_ready_true (test_ccs811.TestCcs811) ... ok
test_read_eco2_tvoc (test_ccs811.TestCcs811) ... ok
test_read_reg (test_ccs811.TestCcs811) ... ok
test_write_reg (test_ccs811.TestCcs811) ... ok
```

```
-----
-----
```

Ran 8 tests in 0.012s

OK